
Explain Like I’m a 500M-Parameter Model: Sequential LLM Knowledge Distillation

Devashish Juyal* Erik Nielsen† Rohan Singh* Krish Kapoor†
juyal@umich.edu erikvn@umich.edu rjsingh@umich.edu krishkap@umich.edu

Abstract

Large language models excel at complex reasoning but are too computationally expensive for deployment on resource-constrained devices. Knowledge distillation can reduce model size, but performance drops sharply when the teacher–student capacity gap is large. We compare sequential and parallel distillation strategies using $7\text{B} \rightarrow 1.5\text{B} \rightarrow 0.5\text{B}$ model stages. Our results show that progressive distillation with properly sized intermediates enables effective knowledge transfer across much larger capacity gaps.

1 Introduction

Large language models (LLMs) have demonstrated remarkable reasoning capabilities across diverse domains such as mathematics, scientific problem-solving, and commonsense inference. However, deploying such models in real-world applications remains challenging due to their immense computational cost and memory footprint. This motivates the need for reasoning distillation, which transfers the reasoning ability of large models into smaller, more efficient students that can operate under tight resource constraints.

A core challenge is that reasoning in LLMs is not only embedded in their final outputs but also distributed across the intermediate chain-of-thought (CoT) steps that connect an input question to its answer. Directly distilling these long rationales is often unstable and computationally heavy. Recent work such as Keypoint-based Progressive Distillation (KPOD) proposes a structured approach: identify key intermediate reasoning points (or keypoints) and train smaller models to imitate them progressively. This idea promises interpretable and modular reasoning transfer, but it also introduces the question of how progressive supervision should be scheduled—sequentially (step-by-step) or in parallel (jointly across keypoints).

While our initial plan was to implement the full KPOD framework, in practice we adopted a logit-level distillation objective for tractability and compute efficiency. Instead of aligning reasoning keypoints, our student models directly matched the teacher’s token-level output distributions while investigating the same sequential vs. parallel training distinction. Our findings show that using a sequential-type distillation method yields better results than parallel distillation.

2 Related Work

Chain-of-Thought Prompting. The ability of large language models (LLMs) to perform multi-step reasoning was popularized by the *chain-of-thought* (CoT) prompting paradigm (10), in which models are explicitly instructed to “think step by step.” Subsequent works demonstrated that intermediate reasoning traces greatly enhance performance on arithmetic and logical tasks such as GSM8K. However, reproducing such reasoning chains in smaller models remains challenging, as they are often long, redundant, and only loosely aligned with model confidence.

*Department of Electrical and Computer Engineering, University of Michigan, Ann Arbor

†Department of Computer Science, University of Michigan, Ann Arbor

Knowledge Distillation. Knowledge distillation (KD) (1) transfers predictive knowledge from a large teacher network to a compact student by minimizing the divergence between their output distributions. This principle has been widely adopted in model compression for NLP. While classical KD focuses on soft-label imitation at the output layer, it does not explicitly address multi-step reasoning or the internal structure of language model thought processes.

Distilling Chain-of-Thought Reasoning. Recent work extends KD to reasoning traces. (12) trains smaller models to imitate the teacher’s generated CoT rationales alongside final answers, effectively treating reasoning as a supervision signal. The idea is to not just memorize an answer, but to understand the thought process that goes along with generating a good answer. (11) propose teaching smaller models to be effective at specific subtasks. Rather than trying to have a student model be generically effective, the idea is to specialize in one particular area (such as mathematical reasoning) at the cost of general capabilities. Despite these advances, direct CoT distillation is often unstable and computationally heavy: longer rationales amplify token-level variance and lead to noisy training.

Progressive and Curriculum Learning. Progressive and curriculum learning (13) provide a conceptual lens for structured reasoning transfer. By organizing supervision signals from simple to complex, these methods improve convergence and generalization across domains. Keypoint-based Progressive Distillation (KPOD) (3) builds on this idea by identifying a minimal set of “key reasoning points” from the teacher’s CoT, training the student to imitate these keypoints in order of logical dependency. This yields interpretable, modular reasoning representations and avoids overfitting to full rationales.

Our Position. Our work draws conceptual motivation from KPOD but adapts it to a more lightweight, logit-based setup suitable for resource-limited training. Rather than distilling explicit reasoning keypoints, we explore how progressive structure, implemented as either *sequential* logit distillation (as opposed to parallel), affects reasoning transfer. This formulation shows that curriculum-style progression (in our case, sequential) can effectively transfer reasoning capabilities.

3 Method

When the parameter ratio between teacher and student is substantial (e.g., 7B $\rightarrow$ 0.5B parameters, representing a 14 $\times$ reduction), direct distillation often fails because the student lacks sufficient capacity to model the teacher’s complex reasoning patterns. This capacity gap manifests in several observable ways: high KL divergence between student and teacher distributions where $D_{KL}(P_S||P_T) \gg 0$, poor generalization on reasoning-heavy tasks, and inability to capture nuanced decision boundaries in the teacher’s output space.

The fundamental issue is that when the representational capacity of the student $\mathcal{F}_S$ cannot adequately approximate the projection of the teacher’s function space $\mathcal{F}_T$ onto the target task, direct distillation fails to preserve the teacher’s capabilities. This theoretical limitation motivates our progressive approach, which decomposes the large capacity gap into multiple manageable steps.

Keypoint-based Progressive Chain-of-Thought Distillation To address the capacity gap problem, we study KPOD, which decomposes the distillation process into manageable stages using intermediate teacher assistants and emphasizes salient reasoning steps through keypoint extraction. The framework operates on two core principles: progressive capacity reduction and keypoint guided learning. In interest of time, we will not discuss KPOD in depth in this report.

3.1 Sequential Distillation Variant

In the sequential variant, models are trained in a cascade: each stage fully converges before becoming the teacher for the next. First, the intermediate model M is trained to mimic the frozen teacher T using soft targets and ground-truth labels, yielding M^* . In the second stage, the frozen M^* teaches the student S under the same objectives, producing the final model S^* .

At each step, the student receives the instruction x , teacher distributions $P_{\text{teacher}}^{(t)}(\cdot | x_{<t}; \tau)$, ground-truth tokens y_t , and keypoint masks m_t , and generates predictions $\hat{y}_t = S(x, \hat{y}_{<t})$. The loss combines teacher supervision and cross-entropy on labels.

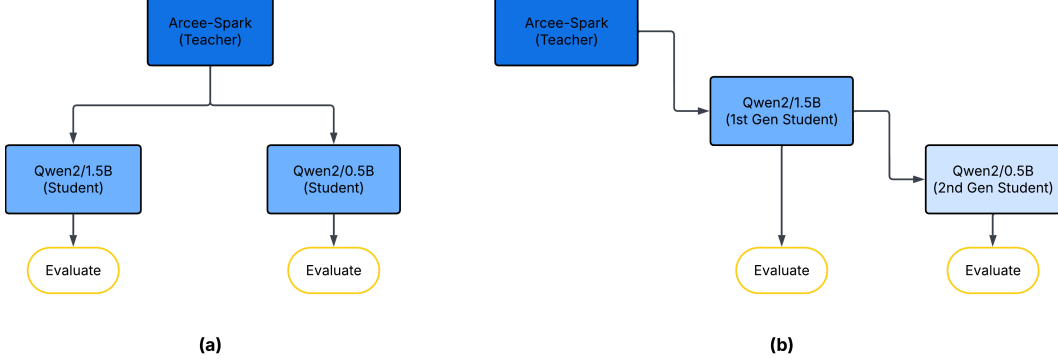


Figure 1: Experimental Setup: (a) Parallel Distillation vs (b) Sequential Distillation

Sequential distillation provides clean stage separation, fully optimized teachers, lower memory usage, and interpretable intermediate checkpoints. Its drawbacks include linear training time, irreversible early-stage errors, and the need to store intermediate models.

3.2 Parallel Distillation Variant

In the parallel variant, the intermediate model M and student S are trained simultaneously. The teacher T remains frozen and supervises M , while both M and S update in the same iteration. Thus, M learns from T while concurrently teaching S , exposing the student to a teacher that is still improving.

The student receives the instruction x and soft targets from the current M , and keypoints are recomputed based on both models' distributions. Unlike the sequential setup, the student observes the intermediate model's entire learning trajectory.

Parallel distillation reduces total training time and enables richer co-evolution, but introduces moving-target instability and risks propagating early-stage errors from M to S .

3.2.1 Multi-Stage Loss Formulation

For a progressive distillation framework with models $\{T, M, S\}$, we formulate the optimization objectives differently depending on whether we employ sequential or parallel training. Sequential distillation performs two independent optimization stages:

$$\text{Stage 1: } \theta_M^* = \theta_M \mathbb{E}_{(x,y) \sim \mathcal{D}} [\mathcal{L}_{\text{total}}(\theta_M; \theta_T, \tau, \alpha_1, \beta_1)] \quad (1)$$

$$\text{Stage 2: } \theta_S^* = \theta_S \mathbb{E}_{(x,y) \sim \mathcal{D}} [\mathcal{L}_{\text{total}}(\theta_S; \theta_M^*, \tau, \alpha_2, \beta_2)] \quad (2)$$

where Stage 2 begins only after θ_M^* has converged, ensuring that the student learns from a stable, high-quality teacher.

Note that gradients flow independently through each model's parameters. The gradient $\nabla_{\theta_M} \mathcal{L}_{\text{total}}(\theta_M; \cdot)$ updates M and $\nabla_{\theta_S} \mathcal{L}_{\text{total}}(\theta_S; \cdot)$ updates S , with no explicit $\frac{\partial \mathcal{L}_S}{\partial \theta_M}$ term. However, M implicitly influences S 's training through its role as teacher, creating the co-evolution dynamic discussed earlier.

4 Experimental Setup

In this section, we describe the experimental configuration, implementation details, and evaluation methodology used in our knowledge distillation framework for large language models.

Packages and Libraries Our distillation pipeline is implemented in PyTorch, leveraging the HuggingFace Transformers library for model loading and inference, and the Transformer Reinforcement

Learning (TRL) library for training orchestration. The implementation extends the DistillKit framework (8) with custom modifications to support multi-stage sequential distillation across models of varying capacities.

Dataset We conduct our experiments using the FineTome-100k dataset (7), a high-quality curated instruction-following corpus formatted in ShareGPT conversation style. To ensure computational tractability while maintaining experimental rigor, we sample 10,000 instances from the full corpus, partitioning them into training and validation sets with a 90/10 split ratio. Each conversational sample is preprocessed using the ChatML template format, which employs special delimiter tokens (`<|im_start|>` and `<|im_end|>`) to explicitly demarcate speaker turns and message boundaries, thereby providing structured context for the language models.

Data Preprocessing Conversational data undergoes a multi-stage preprocessing pipeline. First, conversations in ShareGPT format are transformed into a standardized message structure with explicit role annotations (`system`, `user`, `assistant`). Subsequently, all messages are tokenized using the model-specific tokenizer with a maximum sequence length constraint of 4,096 tokens. Sequences exceeding this threshold are truncated, while shorter sequences are right-padded to the maximum length. During training, padding tokens are masked by assigning them a label value of -100 , ensuring they are excluded from gradient computation and do not contribute to the optimization objective.

Optimizations We employ the AdamW optimizer (?) with a base learning rate of 2×10^{-5} and weight decay coefficient of 0.05 to prevent overfitting. Training proceeds for 3 epochs with a cosine annealing learning rate schedule, incorporating a 10% warmup phase to stabilize early-stage optimization. All experiments happen in bfloat16 mixed-precision training to reduce memory consumption while maintaining numerical stability and convergence properties.

Knowledge Distillation Configuration Following established knowledge distillation methodology (1), we apply temperature scaling to the softmax outputs of both teacher and student models. Specifically, we set the distillation temperature to $\tau = 2.0$ and the distillation coefficient to $\alpha = 0.5$, which balances the knowledge distillation objective against the ground-truth supervision signal. The composite loss function is formulated as:

$$\mathcal{L}_{\text{total}} = \alpha \cdot \mathcal{L}_{\text{KD}} + (1 - \alpha) \cdot \mathcal{L}_{\text{CE}} \quad (3)$$

$$\mathcal{L}_{\text{KD}} = \tau^2 \cdot D_{\text{KL}} \left(\sigma \left(\frac{z_s}{\tau} \right) \parallel \sigma \left(\frac{z_t}{\tau} \right) \right) \quad (4)$$

where the knowledge distillation loss $\mathcal{L}_{\text{KD}}$ is computed as the Kullback-Leibler (KL) divergence between temperature-scaled student and teacher distributions and z_s and z_t denote the logit vectors produced by the student and teacher models respectively, $\sigma(\cdot)$ represents the softmax function, and $\mathcal{L}_{\text{CE}}$ is the standard cross-entropy loss computed against ground-truth labels. The τ^2 scaling factor compensates for the magnitude reduction induced by temperature scaling, ensuring the gradient magnitudes remain comparable to those of the cross-entropy term (1).

Vocabulary Alignment To address potential vocabulary size mismatches between heterogeneous teacher and student architectures, we implement dynamic logit tensor alignment. When the vocabulary dimensions differ between models, the logit tensor with smaller vocabulary size is zero-padded along the vocabulary dimension to match the larger tensor’s shape prior to KL divergence computation. This padding ensures dimensional compatibility while assigning zero probability mass to tokens outside the smaller model’s vocabulary.

Model Architectures We evaluate our distillation framework using models from the Qwen2 family, which provides a range of decoder-only transformer architectures with systematically varying model capacities. Table 1 presents the architectural specifications of the models employed in our experiments.

Table 1: Architectural specifications of Qwen2 model variants used in our experiments.

Model	Parameters	Layers	Hidden Dim	Attention Heads
Qwen2-0.5B	0.5B	24	896	14
Qwen2-1.5B	1.5B	28	1,536	12
Arcee-Spark	7B	32	4,096	32

Teacher Model We adopt Arcee-Spark (arcee-ai/Arcee-Spark) as our teacher model, a 7-billion-parameter instruction-tuned variant based on the Qwen2-7B architecture. Arcee-Spark has undergone additional supervised fine-tuning on high-quality instruction-following datasets, enhancing its capability to generate coherent, contextually appropriate responses. Throughout all distillation experiments, the teacher model parameters remain frozen (i.e., `requires_grad=False`), serving solely as a source of soft targets for student training.

Student Models We train two student models spanning a range of capacities: Qwen2-0.5B (smallest, 500M parameters) and Qwen2-1.5B (medium, 1.5B parameters). To optimize GPU memory utilization and enable experiments on resource-constrained hardware, we adopt a sequential training strategy, processing students in ascending order of size.

5 Results

We evaluate our logit distillation framework by comparing three distillation configurations:

Quantitative Results Table 2 presents comprehensive evaluation metrics across all models. The results demonstrate that sequential progressive distillation substantially outperforms direct distillation across all metrics. The sequential 0.5B model delivers consistently stronger performance than direct distillation, yielding substantial gains across perplexity, cross-entropy loss, accuracy, and KL divergence. Relative to the directly distilled 0.5B model, the sequential approach reduces perplexity by 22.8% (1.76 vs. 2.28), lowers the average loss by 31.5% (0.564 vs. 0.823), and improves Top-1 accuracy by 8.6 percentage points (85.05% vs. 78.30%). These improvements are reflected across all evaluation plots, confirming that progressive distillation provides a more effective learning trajectory for compact student models. Notably, the sequential 0.5B variant even surpasses the 7B teacher in both perplexity and Top-1 accuracy, an outcome attributable to the student’s specialization on the training distribution rather than the teacher’s broader generalization requirements.

Table 2: Evaluation results on held-out test set. Models are trained on same dataset and hyperparams.

Model	Perplexity ↓	Avg Loss ↓	Top-1 ↑	Top-5 ↑	KL Div ↓
Arcee-Spark	2.89	1.063	75.68%	92.73%	—
Qwen2/0.5B	2.28	0.823	78.30%	93.77%	6522.56
Qwen2/1.5B	1.97	0.676	81.30%	95.48%	6379.52
Qwen2/0.5B (2nd Gen)	1.76	0.564	85.05%	96.31%	6072.96

Beyond raw accuracy, the sequentially distilled model also exhibits the closest alignment with the teacher’s output distribution. Its KL divergence (6072.96) represents a 6.9% reduction compared to direct distillation and a 4.8% improvement over the intermediate 1.5B stage, indicating more effective transfer of the teacher’s probabilistic structure. The 1.5B intermediary is central to this improvement: by narrowing the capacity gap in stages (7B → 1.5B → 0.5B), the model avoids the optimization challenges of a single large distillation jump and facilitates smoother knowledge transfer. As a result, the final 0.5B student retains both the teacher’s distributional characteristics and the intermediate model’s refined decision boundaries.

Qualitative Evaluation Our qualitative evaluation compares responses from the 7B teacher and the distilled 0.5B student across explanation, coding, and comparative reasoning prompts, revealing that the student closely mirrors the teacher’s structure, clarity, and technical accuracy despite its significantly smaller size. The student consistently reproduces the teacher’s organized style using

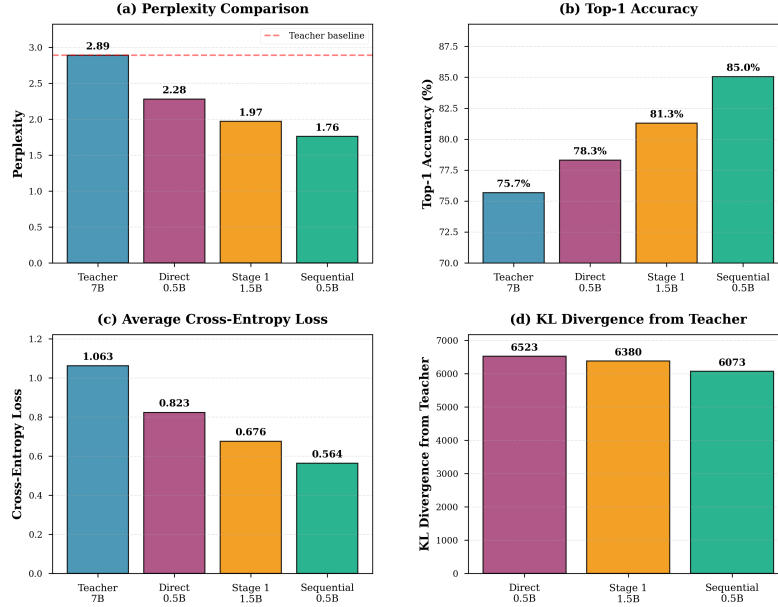


Figure 2: Bar plots of performance metrics of trained LMs

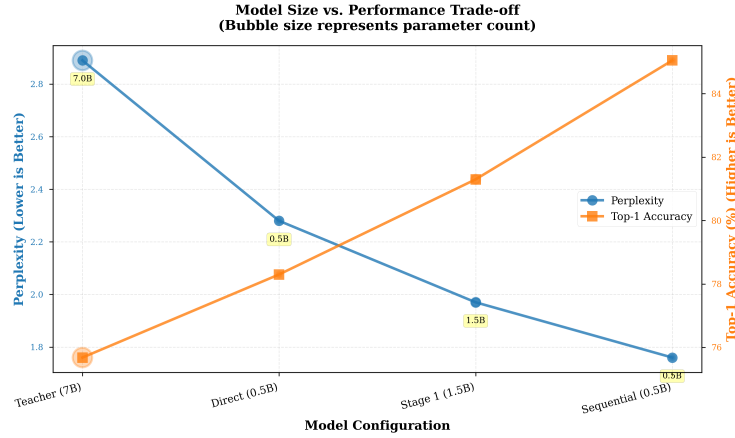


Figure 3: Model Size vs Performance Curves

clear explanations, formatted code blocks, and enumerated lists while maintaining coherence and correctness across tasks. In explanation prompts, the student conveys the same conceptual understanding even when omitting some of the teacher’s analogies; in code generation, it produces functionally correct implementations that follow the teacher’s recursive patterns; and in comparative analysis tasks, it preserves the teacher’s high-level distinctions with only minor differences in phrasing.

6 Conclusion

While our results are limited to a single teacher student family and dataset; broader evaluation across architectures, tasks, and reasoning benchmarks is needed to validate generality. Implementation of keypoint extraction or additional intermediate stages may further improve knowledge transfer. Exploring domain-specific applications, parallel variants, and combination with quantization or pruning offers exciting directions for future work. By enabling high-quality small models suitable for edge deployment, this work contributes toward more accessible and sustainable language model use while retaining strong task-specific performance.

Contributions

- Devashish Juyal: Implemented the KPOD framework from scratch and then setup parallel distillation experimentation pipelines. Also worked on evaluations of the LMs which was another extension of DistillKit that we had to code from scratch, and wrote experiments/results/evals in report.
- Erik Nielsen: Worked on functionalities of KPOD and DistillKit training code and debugging them so they could work within compute and memory constraints of GreakLakes cluster. Facilitated large portion of experiment running on the cluster.
- Rohan Singh: Helped with debugging of training pipeline for compute efficiency, experiment design, wrote up report and intro/related/methods section and poster method / intro sections.
- Krish Kapoor: Created Sequential Distillation architecture from scratch and set up training of sequential distillation experiment. Also worked on project proposal and researched related work to help influence the experiments.

References

- [1] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- [2] Y. Zhang, Y. Ren, J. Ma, *et al.* Cooperative knowledge distillation: A learner-agnostic approach. In *ICML*, 2021.
- [3] S. Feng *et al.* Keypoint-based progressive chain-of-thought distillation for LLMs. *arXiv preprint arXiv:2405.16064*, 2024.
- [4] K. Pillutla *et al.* MAUVE: Measuring the gap between neural text and human text using divergence frontiers. In *NeurIPS*, 2021.
- [5] T. Zhang *et al.* BERTScore: Evaluating text generation with BERT. In *ICLR*, 2020.
- [6] L. Fang *et al.* Knowledge distillation and dataset distillation of large language models: Emerging trends, challenges, and future directions. *arXiv preprint arXiv:2504.14772*, 2025.
- [7] M. Labonne. FineTome-100k: A curated 100k-sample dataset for language model finetuning. *Hugging Face*, 2024. Available at: <https://huggingface.co/datasets/mlabonne/FineTome-100k>.
- [8] ArceeAI. DistillKit: Tools and recipes for efficient LLM distillation. *Hugging Face*, 2024. Available at: <https://huggingface.co/arcee-ai/DistillKit>.
- [9] S. I. Mirzadeh, M. Farajtabar, A. Li, N. Levine, A. Matsukawa, and H. Ghasemzadeh. Improved knowledge distillation via teacher assistant. *arXiv preprint arXiv:1902.03393*, 2020.
- [10] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” *arXiv preprint arXiv:2201.11903*, 2023. Available: <https://arxiv.org/abs/2201.11903>
- [11] Y. Fu, H. Peng, L. Ou, A. Sabharwal, and T. Khot, “Specializing Smaller Language Models towards Multi-Step Reasoning,” *arXiv preprint arXiv:2301.12726*, 2023. Available: <https://arxiv.org/abs/2301.12726>
- [12] L. C. Magister, J. Mallinson, J. Adamek, E. Malmi, and A. Severyn, “Teaching Small Language Models to Reason,” *arXiv preprint arXiv:2212.08410*, 2023. Available: <https://arxiv.org/abs/2212.08410>
- [13] P. Soviany, R. T. Ionescu, P. Rota, and N. Sebe, “Curriculum Learning: A Survey,” *arXiv preprint arXiv:2101.10382*, 2022. Available: <https://arxiv.org/abs/2101.10382>